

## PROJECT TITLE: FILE SYSTEM DEVELOPMENT

**Student Name:** Rakshita Beera Gouda

**Internship Program:** Cybersecurity Internship Program

**Organization:** Centre of Excellence – Digital Forensics and Cybersecurity

**Platform:** Ubuntu Linux (VMware Virtual Machine)

**Date:** 03/02/2026

---

### Table Of Contents

1. Abstract
  2. Problem Statement
  3. Objectives
  4. System Overview
  5. System Architecture
  6. File Storage and Block Allocation System
  7. Implementation and Methodology
  8. User Interface and Output
  9. Test Results and Validation
  10. Conclusion and Future Enhancements
- 

### 1. Abstract

A file system is an essential component of an operating system that manages how data is stored, accessed, and organized. This project focuses on the development of a basic file system using data structures to simulate core file management operations. The system supports file creation, reading, writing, and deletion while maintaining an inode-based structure and a block allocation mechanism. The project is implemented in Python and executed inside an Ubuntu Linux virtual machine using VMware, providing a secure and isolated environment. Through visual representation of file metadata and block usage, the project offers a clear understanding of file allocation and deallocation processes without interacting with physical storage devices.

---

## 2. Problem Statement

File systems provide a structured way to store and retrieve data efficiently. Modern operating systems use advanced file systems that handle large volumes of data, multiple users, and concurrent processes. However, their internal complexity makes them difficult to understand at a beginner or academic level. Studying a simplified file system model helps in understanding the fundamental concepts that form the basis of real-world file systems. Real file systems involve hardware interaction and kernel-level operations, making them unsuitable for introductory learning. There is a need for a simplified, software-based file system model that demonstrates essential file operations and memory management concepts in a controlled and isolated environment.

---

## 3. Objectives

The main objectives of this project are:

- To design a simplified file system using data structures
  - To implement basic file operations such as create, read, write, and delete
  - To simulate file allocation and deallocation mechanisms
  - To represent file metadata using an inode table
  - To execute and test the system in a Linux virtual environment
- 

## 4. System Overview

The developed system simulates a file system entirely within application memory. Files are represented logically using inodes that store metadata such as file name and size. Memory usage is simulated using fixed-size logical blocks, which are allocated and deallocated dynamically based on file operations. The system ensures that storage is allocated only when necessary and released when files are deleted. System logs provide real-time feedback on all operations performed.

---

## 5. System Architecture

The system follows a modular and layered architecture to ensure clarity and maintainability.

### Architecture Layers

- **User Interface Layer:**  
Provides graphical and terminal-based interfaces for user interaction.
  - **File Management Layer:**  
Handles core file operations including create, read, write, and delete.
  - **Storage Simulation Layer:**  
Manages inode entries and block allocation using data structures.
  - **Logging Layer:**  
Records and displays system messages for each operation.
- 

## 6. File Storage and Block Allocation System

The storage system is simulated using fixed-size logical blocks. Each file occupies one or more blocks depending on its size. An inode maintains references to the allocated blocks and stores file metadata.

### Allocation Rules

- Storage is allocated during file creation and write operations
  - Reading a file does not affect storage allocation
  - Storage is released during file deletion
  - Freed blocks are reused efficiently
- 

## 7. Implementation and Methodology

The project was implemented using the following methodology:

1. Installation of Ubuntu Linux inside VMware
2. Configuration of the virtual environment

3. Design of data structures for file and block management
  4. Implementation of file operations (create, read, write, delete)
  5. Integration of inode table and block allocation logic
  6. Implementation of system logging
  7. Testing of all file operations
  8. Validation of allocation and deallocation behaviour
- 

## **8. User Interface and Output**

### **8.1 User Manual**

#### **System Requirements**

- Ubuntu Linux
  - VMware Workstation
  - Python 3.x
  - Linux Terminal
- 

#### **How to Start the Application**

1. Start Ubuntu Linux in VMware.
  2. Open the terminal.
  3. Go to the project folder.
  4. Run the program using `python3 file_system.py`.
  5. The application interface will open.
- 

#### **Using the Application**

##### **i. Create File**

- Click **Create**
- Enter file name and size
- File is added to inode table and blocks are allocated

## ii. Write File

- Select a file
- Click **Write**
- Enter data to store in the file

## iii. Read File

- Select a file
- Click **Read**
- File content is displayed

## iv. Delete File

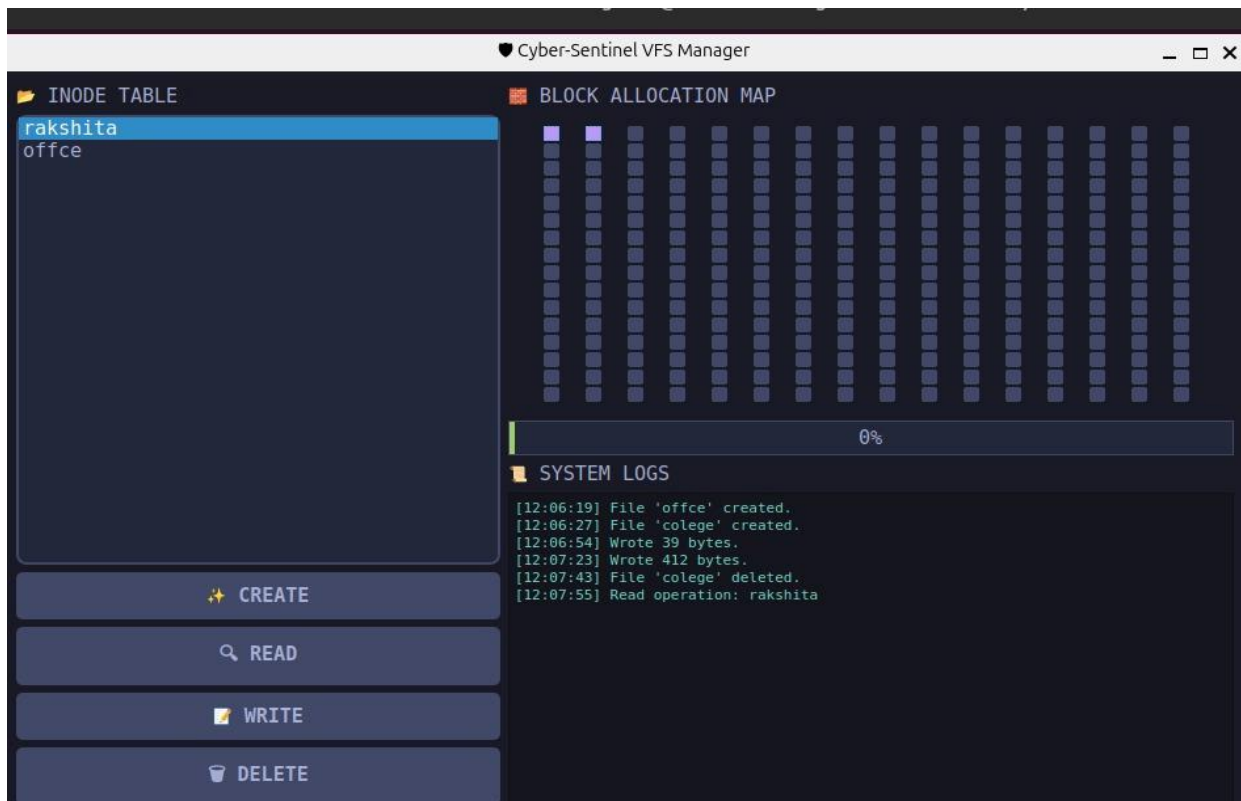
- Select a file
- Click **Delete**
- File is removed and blocks are freed

---

## User Interface

### Graphical User Interface (GUI)

The GUI displays the inode table, block allocation map, and control buttons for file operations. It provides a clear visual representation of memory usage and file metadata.



## Terminal User Interface (TUI)

The TUI allows users to interact with the system through terminal-based commands or menu options, offering a lightweight and efficient interaction method.

```

rakshita-beera-gouda@rakshita-beera-gouda-Latitude-5490: ~/rakshita
rakshita-beera-gouda@rakshita-beera-gouda-Latitude-5490:~$ cd rakshita
rakshita-beera-gouda@rakshita-beera-gouda-Latitude-5490:~/rakshita$ python vfs_engine.py
Virtual File System TUI Active
Commands: create, write, read, delete, list, info, stats, exit
fs> list
Rakshita
fs> stats Rakshita
{'total_blocks': 256, 'used_blocks': 1, 'free_blocks': 255}
fs> info Rakshita
{
  "name": "Rakshita",
  "size": 36,
  "blocks": [
    0
  ],
  "created": "Wed Feb  4 12:14:57 2026",
  "modified": "Wed Feb  4 12:16:09 2026",
  "accessed": "Wed Feb  4 12:16:21 2026",
  "permissions": "rw"
}
fs> write Rakshita "hello!"
Wrote 8 bytes.
fs> read Rakshita
"hello!"
fs> delete Rakshita
File 'Rakshita' deleted.
fs>
```

---

## Exit the Application

- Use the **Exit** option or close the window safely.

---

## 9. Test Results and Validation

The system was tested under multiple scenarios to verify correctness.

| Operation     | Expected Result                    | Status |
|---------------|------------------------------------|--------|
| File Creation | File created with block allocation | Pass   |
| File Writing  | Data stored and blocks updated     | Pass   |
| File Reading  | Data retrieved without allocation  | Pass   |
| File Deletion | Blocks deallocated successfully    | Pass   |

## **10. Conclusion and Future Enhancements**

### **Conclusion**

This project successfully demonstrates the internal working of a basic file system using data structures. By simulating inode management and block allocation in a Linux virtual environment, the project provides practical insight into file system concepts. The implementation strengthens understanding of operating system fundamentals and virtualization.

### **Future Enhancements**

- Support for persistent disk-based storage
- Implementation of file permissions and access control
- Encryption of stored files for enhanced security
- Multi-user file handling
- Performance optimization for larger datasets